

```
import pandas as pd
df=pd.read_csv("Drug_Data.csv")
```

```
df.head()
```

	drugName	Prescribed_for \
0	Mirtazapine	Depression
1	Mesalamine	Crohn's Disease, Maintenance
2	Bactrim	Urinary Tract Infection
3	Contrave	Weight Loss
4	Cyclafem 1 / 35	Birth Control

	Drug_Review	User_Rating
Date \		
0	"I've tried a few antidepressants over th...	10 28-
Feb-12		
1	"My son has Crohn's disease and has done ...	8 17-
May-09		
2	"Quick reduction of symptoms"	9 29-
Sep-17		
3	"Contrave combines drugs that were used for al...	9 05-
Mar-17		
4	"I have been on this birth control for one cyc...	9 22-
Oct-15		

	Count_of_Reviews
0	22
1	17
2	3
3	35
4	4

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53766 entries, 0 to 53765
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   drugName              53766 non-null  object
1   Prescribed_for        53471 non-null  object
2   Drug_Review           53766 non-null  object
3   User_Rating           53766 non-null  int64
4   Date                  53766 non-null  object
5   Count_of_Reviews      53766 non-null  int64
dtypes: int64(2), object(4)
memory usage: 2.5+ MB
```

```
df.describe()
```

	User_Rating	Count_of_Reviews
count	53766.000000	53766.000000
mean	6.976900	27.989752
std	3.285207	36.172833
min	1.000000	0.000000
25%	4.000000	6.000000
50%	8.000000	16.000000
75%	10.000000	36.000000
max	10.000000	949.000000

```
df.columns
```

```
Index(['drugName', 'Prescribed_for', 'Drug_Review', 'User_Rating',
      'Date',
      'Count_of_Reviews'],
      dtype='object')
```

```
df.isnull().sum()
```

```
drugName      0
Prescribed_for 295
Drug_Review    0
User_Rating    0
Date           0
Count_of_Reviews 0
dtype: int64
```

```
df.duplicated()
```

```
0      False
1      False
2      False
3      False
4      False
...
53761  False
53762  False
53763  False
53764  False
53765  False
Length: 53766, dtype: bool
```

```
df.shape
```

```
(53766, 6)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df['Age'].value_counts()
```

```
-----  
-----  
KeyError                                Traceback (most recent call  
last)  
File ~\AppData\Local\Packages\  
PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-  
packages\Python312\site-packages\pandas\core\indexes\base.py:3805, in  
Index.get_loc(self, key)  
    3804 try:  
-> 3805     return self._engine.get_loc(casted_key)  
    3806 except KeyError as err:
```

```
File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()
```

```
File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()
```

```
File pandas\_libs\hashtable_class_helper.pxi:7081, in  
pandas._libs.hashtable.PyObjectHashTable.get_item()
```

```
File pandas\_libs\hashtable_class_helper.pxi:7089, in  
pandas._libs.hashtable.PyObjectHashTable.get_item()
```

```
KeyError: 'Age'
```

The above exception was the direct cause of the following exception:

```
KeyError                                Traceback (most recent call  
last)  
Cell In[13], line 1  
----> 1 df['Age'].value_counts()
```

```
File ~\AppData\Local\Packages\  
PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-  
packages\Python312\site-packages\pandas\core\frame.py:4102, in  
DataFrame.__getitem__(self, key)  
    4100 if self.columns.nlevels > 1:  
    4101     return self._getitem_multilevel(key)  
-> 4102 indexer = self.columns.get_loc(key)  
    4103 if is_integer(indexer):  
    4104     indexer = [indexer]
```

```
File ~\AppData\Local\Packages\  
PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-  
packages\Python312\site-packages\pandas\core\indexes\base.py:3812, in  
Index.get_loc(self, key)  
    3807     if isinstance(casted_key, slice) or (  
    3808         isinstance(casted_key, abc.Iterable)  
    3809         and any(isinstance(x, slice) for x in casted_key)  
    3810     ):  
    3811         raise InvalidIndexError(key)  
-> 3812     raise KeyError(key) from err
```

```

3813 except TypeError:
3814     # If we have a listlike key, _check_indexing_error will
raise
3815     # InvalidIndexError. Otherwise we fall through and re-
raise
3816     # the TypeError.
3817     self._check_indexing_error(key)

```

```

KeyError: 'Age'

```

```

df['BP'].unique()

```

```

array(['HIGH', 'LOW', 'NORMAL'], dtype=object)

```

```

df['Cholesterol'].unique()

```

```

array(['HIGH', 'NORMAL'], dtype=object)

```

```

df['Drug'].unique()

```

```

array(['DrugY', 'drugC', 'drugX', 'drugA', 'drugB'], dtype=object)

```

```

df['Sex'].value_counts().unique()

```

```

array([104, 96])

```

```

df['Na_to_K'].unique()

```

```

array([25.355, 13.093, 10.114, 7.798, 18.043, 8.607, 16.275, 11.037,
      15.171, 19.368, 11.767, 19.199, 15.376, 20.942, 12.703, 15.516,
      11.455, 13.972, 7.298, 25.974, 19.128, 25.917, 30.568, 15.036,
      33.486, 18.809, 30.366, 9.381, 22.697, 17.951, 8.75 , 9.567,
      11.014, 31.876, 14.133, 7.285, 9.445, 13.938, 9.709, 9.084,
      19.221, 14.239, 15.79 , 12.26 , 12.295, 8.107, 13.091, 10.291,
      31.686, 19.796, 19.416, 10.898, 27.183, 18.457, 10.189, 14.16 ,
      11.34 , 27.826, 10.091, 18.703, 29.875, 9.475, 20.693, 8.37 ,
      13.303, 27.05 , 12.856, 10.832, 24.658, 24.276, 13.967, 19.675,
      10.605, 22.905, 17.069, 20.909, 11.198, 19.161, 13.313, 10.84 ,
      13.934, 7.761, 9.712, 11.326, 10.067, 13.935, 13.597, 15.478,
      23.091, 17.211, 16.594, 15.156, 29.45 , 29.271, 15.015, 11.424,
      38.247, 25.395, 35.639, 16.725, 11.871, 12.854, 13.127, 8.966,
      28.294, 8.968, 11.953, 20.013, 9.677, 16.85 , 7.49 , 6.683,
      9.17 , 13.769, 9.281, 18.295, 9.514, 10.103, 10.292, 25.475,
      27.064, 17.206, 22.456, 16.753, 12.495, 25.969, 16.347, 7.845,
      33.542, 7.477, 20.489, 32.922, 13.598, 25.786, 21.036, 11.939,
      10.977, 12.894, 11.343, 10.065, 6.269, 25.741, 8.621, 15.436,
      9.664, 9.443, 12.006, 12.307, 7.34 , 8.151, 8.7 , 11.009,
      7.261, 14.642, 16.724, 10.537, 11.227, 22.963, 10.444, 12.923,
      10.443, 9.945, 12.859, 28.632, 19.007, 26.645, 14.216, 23.003,
      11.262, 12.879, 10.017, 17.225, 18.739, 12.766, 18.348, 10.446,
      19.011, 15.969, 15.891, 22.818, 13.884, 11.686, 15.49 , 37.188,

```

```
25.893, 9.849, 10.403, 34.997, 20.932, 18.991, 8.011, 16.31 ,  
6.769, 34.686, 11.567, 9.894, 14.02 , 11.349])
```

```
from sklearn.preprocessing import LabelEncoder  
le=LabelEncoder()  
df['Sex']=le.fit_transform(df['Sex'])
```

```
df
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	8	0	HIGH	HIGH	25.355	DrugY
1	30	1	LOW	HIGH	13.093	drugC
2	30	1	LOW	HIGH	10.114	drugC
3	12	0	NORMAL	HIGH	7.798	drugX
4	44	0	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	39	0	LOW	HIGH	11.567	drugC
196	1	1	LOW	HIGH	12.006	drugC
197	35	1	NORMAL	HIGH	9.894	drugX
198	8	1	NORMAL	NORMAL	14.020	drugX
199	24	0	LOW	NORMAL	11.349	drugX

```
[200 rows x 6 columns]
```

```
df['BP']=le.fit_transform(df['BP'])
```

```
df
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	8	0	0	HIGH	25.355	DrugY
1	30	1	1	HIGH	13.093	drugC
2	30	1	1	HIGH	10.114	drugC
3	12	0	2	HIGH	7.798	drugX
4	44	0	1	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	39	0	1	HIGH	11.567	drugC
196	1	1	1	HIGH	12.006	drugC
197	35	1	2	HIGH	9.894	drugX
198	8	1	2	NORMAL	14.020	drugX
199	24	0	1	NORMAL	11.349	drugX

```
[200 rows x 6 columns]
```

```
df['Drug']=le.fit_transform(df['Drug'])
```

```
df
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	8	0	0	HIGH	25.355	0
1	30	1	1	HIGH	13.093	3
2	30	1	1	HIGH	10.114	3
3	12	0	2	HIGH	7.798	4

4	44	0	1	HIGH	18.043	0
...	...	...	...	...	...	...
195	39	0	1	HIGH	11.567	3
196	1	1	1	HIGH	12.006	3
197	35	1	2	HIGH	9.894	4
198	8	1	2	NORMAL	14.020	4
199	24	0	1	NORMAL	11.349	4

[200 rows x 6 columns]

```
df['Cholesterol']=le.fit_transform(df['Cholesterol'])
```

df

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	8	0	0	0	25.355	0
1	30	1	1	0	13.093	3
2	30	1	1	0	10.114	3
3	12	0	2	0	7.798	4
4	44	0	1	0	18.043	0
...	...	...	...	...	...	...
195	39	0	1	0	11.567	3
196	1	1	1	0	12.006	3
197	35	1	2	0	9.894	4
198	8	1	2	1	14.020	4
199	24	0	1	1	11.349	4

[200 rows x 6 columns]

```
x = df.drop("Drug", axis=1)
```

```
y=df['Drug']
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
model=RandomForestClassifier(n_estimators=100,random_state=42)
```